Практическая работа №12.

Триггеры. Часть 1.

Триггер – это некое правило, побуждающее действие. С триггером студенты знакомятся еще в курсе теории информационных процессов и систем. Триггеры – это хранимые процедуры специального вида, которые автоматически срабатывают до или после заданного события – INSERT, UPDATE, DELETE. Триггер создается для одной таблицы, но может использовать данные из разных таблиц.

Можно обратиться к столбцам в подчиненной таблице (таблице, связанной с вызывающей), используя псевдонимы OLD и NEW. OLD.col_name обращается к столбцу существующей строки прежде, чем она модифицируется или удалится. NEW.col_name обращается к столбцу новой строки, которая будет вставлена, или же к существующей строке после того, как она модифицируется.

Создание триггера имеет следующий синтаксис:

```
DELIMITER |
CREATE TRIGGER [TRIGGER_NAME]
[TRIGGER TIME] [TRIGGER EVENT]
ON [TABLE]
FOR EACH ROW
[TRIGGER BODY]//
DELIMITER ;*
```

Где:

DELIMITER //: разделитель MySQL по умолчанию — это ;. Его нужно заменить на что-то другое, для того, чтобы MySQL рассматривал следующие строки, как одну команду, пока не достигнет пользовательского разделителя. В данном примере в качестве разделителя используется //, а стандартный разделитель ; стоит в конце.

[TRIGGER_NAME]: триггер должен иметь имя, и вы можете указать его именно здесь.

[TRIGGER TIME]: триггер может быть вызван в разные моменты времени. MySQL позволяет определить, когда запускать триггер — до или после операции с базой данных.

[TRIGGER EVENT]: триггеры могут быть вызваны только операциями INSERT, UPDATE и DELETE. Вы можете использовать любое из значений в зависимости от того, чего вы хотите достичь.

[TABLE]: любой триггер, который вы создаете в своей базе данных MySQL, должен быть связан с таблицей.

FOR EACH ROW: этот оператор позволяет MySQL выполнять код триггера для каждой строки, на которую влияет триггер.

[TRIGGER BODY]: код, который выполняется при вызове триггера, называется *trigger body*. Это может быть один SQL-оператор или несколько команд. Обратите внимание, если вы выполняете несколько SQL-операторов в теле триггера, вы должны заключить их в блок BEGIN... END.

Триггер может происходить после или до событий, описанных выше.

Пример. Триггер после вставки.

В таблице «Пассажиры» добавляются сведения о пассажирах, однако их возраст, скидка на билет и комментарий является ли пассажир совершеннолетним отсутствуют. Для того, чтобы автоматически рассчитывался возраст и скидка для нового пассажира можно написать триггер. Однако, существует проблема – если триггер срабатывает для той же таблицы, для которой происходит событие возникает ошибка целостности данных. Поэтому новый триггер будет срабатывать после добавления данных в таблицу register (Регистрация).

После добавления новой регистрации на рейс автоматически производится подсчет возраста и скидки, которая полагается пассажиру.

```
DELIMITER |
CREATE TRIGGER `update_age` AFTER INSERT ON `register`
FOR EACH ROW
BEGIN
   update passengers
   set age=timestampdiff(year, date_of_birth, now()),

   discount=CASE
   when age<2 then 100
   when age>=2 and aage<18 then 40
   else 0
   end
   where age is null or discount is null;
end;
```

Рисунок 1 – Создание триггера, срабатывающего после вставки.

Пример. Триггер после обновления.

При этом таблица Пассажиры обновляется, а строчка с ценой билета в таблице регистрации остается пустой. Чтобы убрать эту неопределенность необходимо описать триггер, который будет рассчитывать стоимость билета. Оптимально было бы, чтобы цена рассчитывалась после обновлений в таблице Пассажиры, т.к. информация о скидке поступает оттуда. Стоит помнить, что, если данные для расчета берутся из нескольких таблиц, то необходимо подключить внутреннее/левое/правое соединение, иначе данные будут взяты недостоверные.

```
DELIMITER |
create TRIGGER up_price after update on passengers
for EACH ROW
BEGIN
update register inner join avia on register.id_avia=avia.id_avia
inner join city on avia.id_city=city.id_city
inner join passengers on register.id_passenger=passengers.id_pass
set price=CASE
when passengers.discount=0 then 0
when passengers.discount=40 then city.distance*4-city.distance*4*0.4
else city.distance*4
end;
end;
```

Рисунок 2 – Создание триггера, срабатывающего после обновления.

Пример. Триггер после удаления.

Предположим, что пассажира могли занести в «черный список» авиакомпаний или он мог умереть. Поэтому логично, что он будет удален из таблицы Пассажиры, однако некорректно было бы полностью исключить данные о его полетах, т.к. он платил деньги за перелеты и в дальнейшем можно отследить доходность полетов и т.д., поэтому было бы некорректно выбрасывать данные об оплатах даже в такой ситуации. Для сохранности при удалении данных о пассажире в таблице регистраций значение id пассажира станет пустым.

```
DELIMITER |
Create trigger up_pass after delete on passengers
for each ROW
BEGIN
UPDATE register set id_passenger= null;
end;
```

Рисунок 3 – Создание триггера, срабатывающего после удаления.

Для просмотра всей информации о триггерах можно воспользоваться скриптом SHOW TRIGGERS

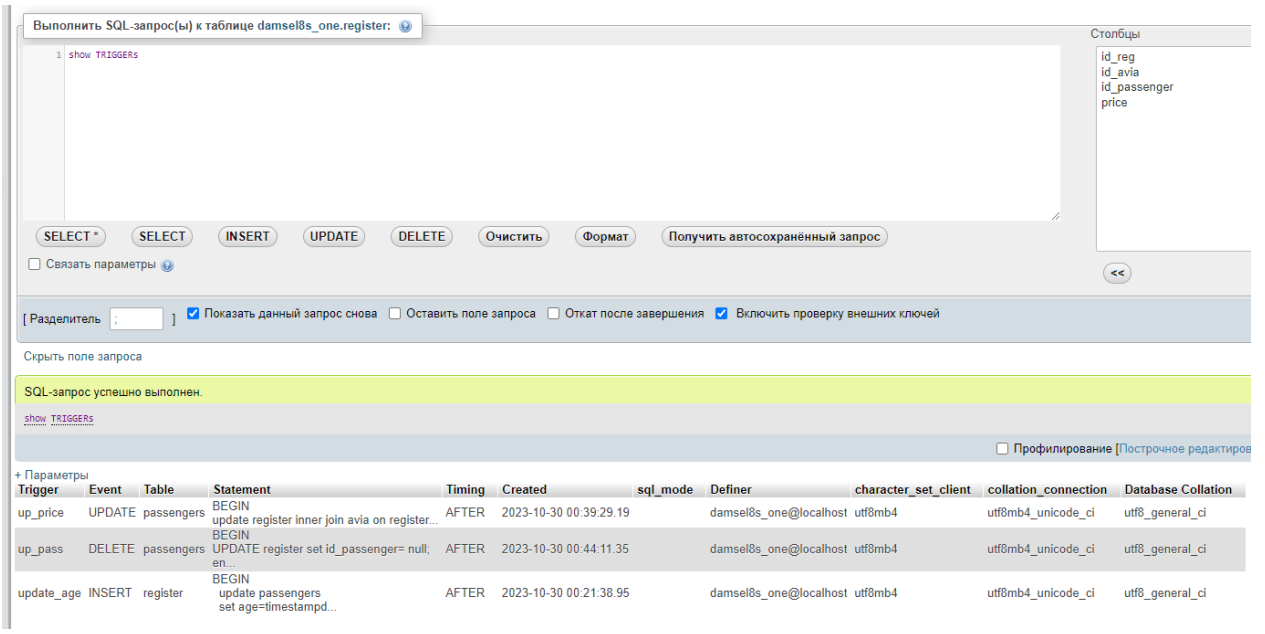


Рисунок 4 – Общий вывод всех триггеров.

Также во вкладке «Триггеры» для каждой таблицы отображается содержание триггеров и возможность их редактировании.

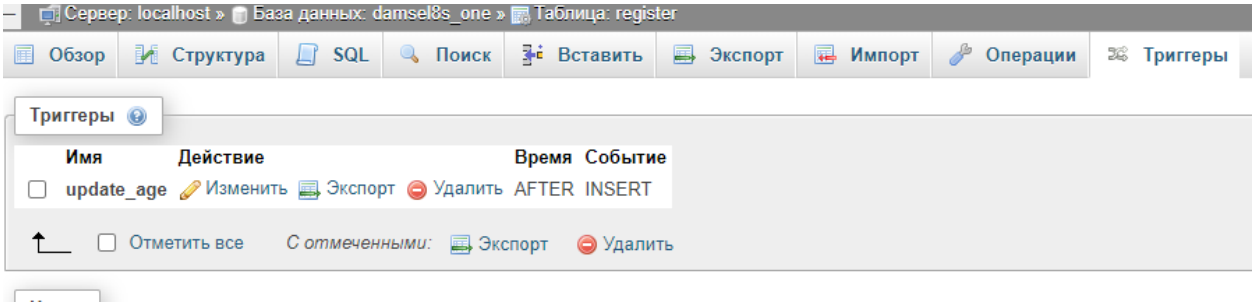


Рисунок 5 – Триггеры для каждой таблицы.

Пример. Триггер логов.

Нередко триггеры используют для того, чтобы хранить логи, которые происходили с той или иной таблицей. Структура такого триггера выглядит следующим образом:

```
DELIMITER |
CREATE TRIGGER `update_test` AFTER INSERT ON `test`
FOR EACH ROW BEGIN
INSERT INTO log Set msg = 'insert', row_id = NEW.id;
END;
```

После каждой вставки в таблицу логов (в данном случае “test”) будет добавляться с новым идентификатором и действием (в данном случае вставка).

Задание 1.

Создать 2 триггера, срабатывающих после удаления, 2 триггера, срабатывающих после вставки и 2 триггера, срабатывающих после обновления. Вывести всю информацию о триггерах в отчете. 2 триггера из указанных сделайте с занесением в таблицу логов. Использовать в триггерах псевдоним NEW.